

Week 1 Module: Programming practice (3rd year)

- Language Used: Java

Star Pattern

1. Square

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

Logic

1. Start the program.
2. Read the number of rows from the user.
3. Use an outer loop to control the number of rows to be printed.
4. For each row, use an inner loop to print * symbols equal to the number of rows.
5. After the inner loop completes, print a new line to move to the next row.
6. Repeat the process until all rows are printed, forming a square pattern.
7. End the program.

Time Complexity

- $O(n^2)$
 - The outer loop runs n times and the inner loop also runs n times therefore, time complexity is $n * n = n^2$.

Space Complexity

- $O(1)$
 - The program doesn't require extra spaces hence the space is constant.

2. Hollow-Square

```
* * * * *
*       *
*       *
*       *
*       *
* * * * *
```

Logic

1. Start the program.
2. Read the number of rows from the user.

3. Use an outer loop to iterate through each row.
4. For each row:
 - If it is the **first** or **last** row, print * for all columns to form the top and bottom borders.
 - Otherwise, use an inner loop to iterate through each column:
 - Print * if it is the **first** or **last** column.
 - Print a space " " for all other columns to create the hollow portion.
5. After printing each row, move to the next line.
6. Repeat until all rows are printed, forming a hollow square pattern.
7. End the program.

Time Complexity

- $O(n^2)$
 - The outer loop runs n times and the inner loop also runs n times even if it is hollow, therefore, time complexity is $n*n = n^2$.

Space Complexity

- $O(1)$
 - The program doesn't require extra spaces hence the space is constant.

3. Triangle

```
*
* *
* * *
* * * *
* * * * *
```

Logic

1. Start the program.
2. Read the number of rows from the user.
3. Use an outer loop to iterate through each row.
4. For each row, use an inner loop to print * symbols equal to the current row number ($i + 1$).
5. After printing the stars for a row, move to the next line.
6. Repeat the process until all rows are printed, forming a right-angled triangle pattern.
7. End the program.

Time Complexity

- $O(n^2)$
 - The outer loop runs n times and the inner runs $1+2+3+ \dots + n = n(n+1)/2$ times which is basically n^2 therefore, time complexity is $n*n = n^2$.

Space Complexity

- $O(1)$
 - The program doesn't require extra spaces hence the space is constant.

4. Inverted-Triangle

```
* * * * *
* * * *
* * *
* *
*
```

Logic

1. Start the program.
2. Read the number of rows from the user.
3. Use an outer loop to iterate through each row.
4. For each row, use an inner loop to print * symbols. The number of stars starts from the total number of rows and decreases by one in each subsequent row.
5. After printing the stars for a row, move to the next line.
6. Repeat the process until all rows are printed, forming an inverted right-angled triangle pattern.
7. End the program.

Time Complexity

- $O(n^2)$
 - The outer loop runs n times and the inner runs $n, n-1, n-2, n-3, \dots, 1$ which is basically n^2 therefore, time complexity is $n*n = n^2$.

Space Complexity

- $O(1)$
 - The program doesn't require extra spaces hence the space is constant.

5. Pyramid

```
      *
     * * *
    * * * * *
   * * * * * * *
  * * * * * * * *
 * * * * * * * * *
```

Logic

1. Start the program.
2. Read the number of rows from the user.
3. Use an outer loop to iterate through each row from 1 to the input value.
4. For each row:
 - Print the required number of leading spaces so that the stars are centered. The number of spaces decreases with each row.
 - Print $2 \times \text{row} - 1$ stars using an inner loop. The number of stars increases by two in every successive row.
5. After printing the spaces and stars for a row, move to the next line.
6. Repeat the process until all rows are printed, forming a pyramid pattern.
7. End the program.

Time Complexity

- $O(n^2)$
 - The outer loop runs n times and the two inner loop runs $2n-1$ per row for printing spaces and stars which is basically n^2 , therefore, time complexity is $n*n = n^2$.

Space Complexity

- $O(1)$
 - The program doesn't require extra spaces hence the space is constant.

6. Diamond

```
      *
     * * *
    * * * * *
   * * * * * * *
  * * * * * * * *
 * * * * * * * * *
* * * * * * * * * *
 * * * * * * * *
  * * * * *
   * * *
    * *
     *
```

Logic

1. Start the program.
2. Read the number of rows from the user.
3. Print the **upper half** of the diamond:
 - Use an outer loop to iterate from 1 to the input value.
 - For each row, print the required leading spaces to centre the pattern.
 - Print $2 \times \text{row} - 1$ stars to form the pyramid.
 - Move to the next line.
4. Print the **lower half** of the diamond:
 - Use another outer loop to iterate from the input value down to 1.
 - For each row, print the required leading spaces, increasing with each iteration.
 - Print $2 \times \text{row} - 1$ stars, decreasing with each iteration.
 - Move to the next line.
5. Repeat until both halves are printed, forming a complete diamond pattern.
6. End the program.

Time Complexity

- $O(n^2)$
 - The outer loop runs n times and the two inner loop runs $2n-1$ per row for printing spaces and stars. This is done through two approaches one to print the upper triangle and one to print the lower which runs at $(2 * n^2)$ which is basically n^2 therefore, time complexity is $n * n = n^2$.

Space Complexity

- $O(1)$
 - The program doesn't require extra spaces hence the space is constant.

(All the pattern programs in module 1 have time complexity of $O(n)$ or $O(n^2)$ with the same logic as above)

Week 2 Module: Programming practice

- Language Used: Java

1. Fraud Transaction Detection (Linear Search)

Problem Statement

A bank stores transaction IDs in the order they occur. Since the transactions are not sorted, the security team needs to check if a suspicious transaction ID exists.

Write a program using Linear Search.

Input Format

- First line contains integer N
- Second line contains N transaction IDs
- Third line contains suspicious transaction ID

Logic

8. Start the program.
9. Read the number of transaction IDs from the user.
10. Read All Transaction IDs into an array.
11. Read the suspicious transaction ID from the user.
12. Use a loop to iterate the array and find the suspicious ID.
13. If the ID is found:
 - Print Fraud Transaction Found.
- Else:
 - Print transaction not found.
14. End the program.

Time Complexity

- $O(n)$
 - The loop runs one time and searches the target key from the data structure hence time complexity is $O(n)$ for worst and average case and $O(1)$ for best case.

Space Complexity

- $O(1)$
 - The program doesn't require extra spaces hence the space is constant.

2. Server Log Search (Binary Search)

Problem Statement

A cloud company stores server log IDs in sorted order. Engineers need to quickly verify if a log ID exists.

Write a program using Binary Search.

Input Format

- First line contains N
- Second line contains sorted log IDs
- Third line contains target log ID

Logic

8. Start the program.
 9. Read the number of server log IDs from the user.
 10. Read the sorted log IDs into an array.
 11. Read the target log ID from the user.
 12. Use a while loop to iterate through the array.
 13. Find the middle element in the array:
 - If the target ID is greater than middle element:
 - Then shorten array starting from the middle+1 element till the array length
 - If the target ID is less than the middle element:
 - Then shorten the array length to the middle element-1.
- Repeat this until the middle element is the targeted ID.
14. If the target ID is found then print "Log is Found" else print "Log not found".
 15. End the program.

Time Complexity

- Best Case: $O(1)$
- Average case: $O(\log n)$
- Worst Case: $O(n \log n)$
 - The binary search divides the searching area repeatedly hence $O(\log n)$.

Space Complexity

- $O(1)$
 - The program doesn't require extra spaces hence the space is constant.

3. Sort Employee Salaries (Bubble Sort)

Problem Statement

A company wants to sort employee salaries in ascending order for payroll analysis.

Write a program using Bubble Sort.

Input Format

- First line contains N
- Second line contains salaries

Output Format

Print sorted salaries.

Logic

8. Start the program.
9. Read the number of employees from the user.
10. Read all the salaries into an array.
11. Use an outer loop to iterate through the salaries.
12. Use an inner loop to swap salaries
 1. If a salary is greater than the next salary in the array then swap those salaries else continue the loop
13. Repeat the process until array is sorted.
14. End the program.

Time Complexity

- Best Case: $O(1)$
- Average case: $O(n^2)$
- Worst Case: $O(n^2)$
 - The bubble sort compares and swaps adjacent element repeatedly hence performing n^2 comparisons resulting in $O(n^2)$ average case. The swapped element checks if a swap has been performed reducing checking time hence $O(n)$ for best case.

Space Complexity

- $O(1)$
 - The program doesn't require extra spaces hence the space is constant.

4. Sort Website Response Time (Insertion Sort)

Problem Statement

A web monitoring system records website response times. New data comes continuously and must be inserted into the correct position.

Write a program using **Insertion Sort**.

Input Format

- First line contains N

- Second line contains response times

Output Format

Print sorted response times.

Logic

8. Start the program.
9. Read the number of response time from the user.
10. Read all the response time into an array.
11. Use an outer for loop to iterate through the array.
12. Use a while loop to divide the array into sorted and unsorted part.
13. Continuously check in the while for a sorted element with unsorted element
 1. If the unsorted element is smaller than the sorted element then place the unsorted element into the correct sorted order.
 2. Else continue.
 3. Do this until array is sorted.
14. End the program.

Time Complexity

- Best Case: $O(n)$
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$
 - Insertion Sort builds the sorted portion of the list one element at a time by shifting larger elements to the right. If the list is already sorted, each element is checked once, resulting in $O(n)$ time.

Space Complexity

- $O(1)$

5. Prioritize Bug Reports (Selection Sort)

Problem Statement

A software company assigns priority scores to bug reports. To resolve bugs efficiently, they want to sort priorities in ascending order.

Write a program using **Selection Sort**.

Input Format

- First line contains N

- Second line contains priority scores

Output Format

Print sorted priority scores.

Logic

8. Start the program.
9. Read the number of priority scores from the user.
10. Read all the scores into an array.
11. Use an outer loop to iterate through the array.
12. Use an inner loop to find min element and swap it at the start, sorting the array from the start
13. Repeat the process until array is sorted.
14. End the program.

Time Complexity

- Best Case: $O(n^2)$
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$
 - Selection Sort repeatedly scans the unsorted portion of the list to find the minimum element and places it in its correct position. Regardless of the initial order of the elements, it always performs approximately $n(n - 1)/2$ comparisons, resulting in $O(n^2)$ time.

Space Complexity

- $O(1)$

Week 3 Module: Programming practice

- Language Used: Java

1. Question

A university's Training and Placement Cell have successfully completed its annual placement drive. The placement department has collected the salary packages (in LPA) offered to students by different companies. However, the records were entered in the order they were received, resulting in an unsorted list of package values.

Before publishing the placement statistics report, the department wants all package values arranged in ascending order. Since the university plans to handle placement data for thousands of students in the coming years, they want a sorting technique that is efficient for large datasets.

Your task is to implement the **Merge Sort algorithm** to sort the package values in increasing order. Merge Sort follows a divide-and-conquer approach by dividing the dataset into smaller subarrays, sorting them recursively, and then merging them back together.

The final sorted list will help the university identify minimum, median, and maximum packages while generating analytical reports.

Input

First line contains an integer **N**, representing the number of placed students.

Second line contains **N space-separated integers**, representing package values (multiplied by 100 to avoid decimals).

Output

Print the package values in ascending order.

Constraints

$$1 \leq N \leq 100000$$

$$100 \leq \text{Package} \leq 10000$$

Logic

1. Start.
2. If the array is null or contains less than 2 elements, return.
3. Find the middle index of the array.
4. Divide the array into two subarrays:
 - Left subarray from index 0 to mid - 1.
 - Right subarray from index mid to the last index.
5. Recursively apply Merge Sort on the left subarray.
6. Recursively apply Merge Sort on the right subarray.
7. Merge the two sorted subarrays:
 - Initialize three indices i, j, and k to 0.
 - Compare the current elements of both subarrays.
 - Copy the smaller element into the original array.
 - Increment the corresponding index and repeat until one subarray is exhausted.

8. Copy the remaining elements of the left subarray, if any, into the original array.
9. Copy the remaining elements of the right subarray, if any, into the original array.
10. The original array is now sorted.
11. Stop.

Time Complexity

- Best Case: $O(n \log n)$
- Average Case: $O(n \log n)$
- Worst Case: $O(n \log n)$
 - Merge Sort recursively divides the array into two halves until each subarray contains a single element. During the merge step, each element is processed exactly once at every level of recursion. Since there are $\log n$ levels and each level processes n elements, the overall time complexity is $O(n \log n)$.

Space Complexity

- $O(n)$

2. Question (Easy)

A cloud infrastructure company manages hundreds of servers distributed across different geographical locations. Every hour, each server reports its average response time in milliseconds. The collected data is stored in the order it arrives and is not sorted.

To generate performance reports, the operations team wants the response times arranged from the fastest server to the slowest server. Since the data size can become very large, an efficient in-place sorting technique is required.

Your task is to implement **Quick Sort** to sort the response times in ascending order. Built-in sorting methods are not allowed.

After sorting, the operations team will use the report to identify high-latency servers and optimize system performance.

Input

- First line contains integer N .
- Second line contains N space-separated response times.

Output

Print the response times in ascending order.

Constraints

$$1 \leq N \leq 100000$$

$1 \leq \text{Response Time} \leq 10000$

Logic

1. Start the program.
2. Read the number of employees from the user.
3. Read all the salaries into an array.
4. Use an outer loop to iterate through the salaries.
5. Use an inner loop to swap salaries
 1. If a salary is greater than the next salary in the array then swap those salaries else continue the loop
6. Repeat the process until array is sorted.
7. End the program.

Time Complexity

- **Best Case: $O(n \log n)$**
- **Average Case: $O(n \log n)$**
- **Worst Case: $O(n^2)$**
 - Quick Sort partitions the array around a pivot and recursively sorts the two subarrays. In the best and average cases, the pivot divides the array into nearly equal halves, resulting in $O(n \log n)$ time. If the pivot is always the smallest or largest element (such as when the array is already sorted and the last element is chosen as the pivot), the recursion becomes unbalanced, leading to $O(n^2)$ time.

Space Complexity

- **Best/Average Case: $O(\log n)$**
- **Worst Case: $O(n)$**
 - The recursive calls require $O(\log n)$ stack space on average and $O(n)$ in the worst case.

1.Question (Easy)

An online gaming platform is hosting a national-level tournament involving thousands of participants. At the end of the tournament, player scores are collected from multiple game servers and stored in random order.

Before announcing the final rankings, the organizers need all scores sorted in ascending order. Since the leaderboard data can be very large, they have decided to use **Heap Sort**, which provides guaranteed $O(N \log N)$ performance.

Your task is to implement Heap Sort and arrange all player scores in increasing order.

The sorted output will help tournament officials generate rankings and determine qualification thresholds for future events.

Input

- First line contains integer N.
- Second line contains **N space-separated player scores**.

Output

Print the scores in ascending order.

Constraints

$$1 \leq N \leq 100000$$

$$0 \leq \text{Score} \leq 1000000$$

Input Format

- First line contains N
- Second line contains priority scores
- Output Format
- Print sorted priority scores.

Logic

1. Start.
2. Read the number of scores from the user.
3. Create an array to store the scores.
4. Input all the scores into the array.
5. Convert the array into a Max Heap.
6. Repeat until the heap contains only one element:
 - Swap the root element (largest element) with the last element of the heap.
 - Reduce the heap size by one.
 - Restore the Max Heap property by heapifying the root.
7. Stop.

Time Complexity

- **Best Case: $O(n \log n)$**
- **Average Case: $O(n \log n)$**
- **Worst Case: $O(n \log n)$**
 - Heap Sort first builds a Max Heap in $O(n)$ time. It then repeatedly removes the largest element by swapping the root with the last element and restoring the heap

using heapify(), which takes $O(\log n)$ for each of the n elements. Therefore, the overall time complexity is $O(n \log n)$.

Space Complexity

- **$O(1)$** (excluding recursion stack)
- **$O(\log n)$** if the recursive heapify() call stack is counted.

Problem Statement:

Given a positive integer N , print all the numbers from 1 to N in increasing order, separated by a single space.

Your task is to write a program that prints the required sequence.

After completing the program, analyze your solution by calculating its Time Complexity and Space Complexity, and explain why your answer is correct.

Input Format

A single integer N .

Output Format

Print all integers from 1 to N in increasing order separated by spaces.

Constraints

$$1 \leq N \leq 10^5$$

Logic

1. Start.
2. Read n from user.
3. Using a for loop print numbers from 1 to n .
4. Stop.

Source Code

```
import java.util.Scanner;

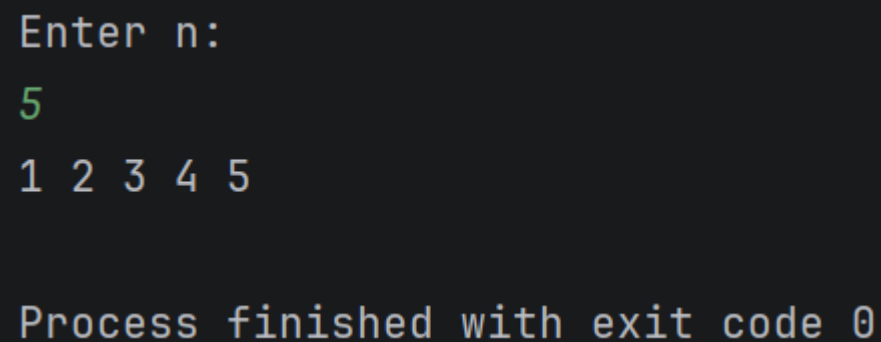
import static problemOne.ProblemOne.*;

public class Main {
    public static void main(String[] args) {
        System.out.println("Enter n: ");
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        printNNumbers(n);
    }
}
```

```
package problemOne;

public class ProblemOne {
    public static void printNNumbers(int n) {
        for (int i = 1; i <= n; i++) {
            System.out.print(i+" ");
        }
        System.out.println();
    }
}
```

Output



```
Enter n:
5
1 2 3 4 5

Process finished with exit code 0
```

Time Complexity

- $O(n)$
 - As there is only one loop which runs till n the time complexity is $O(n)$.

Space Complexity

- $O(1)$
 - No extra space is required

Problem Statement:

Given two integers A and B, calculate the value of A^B (A raised to the power B) using a loop.

Your task is to write a program that computes the result and prints it.

Once your program is complete, analyze your solution by calculating its Time Complexity and Space Complexity, and explain why your answer is correct.

Input Format

Two integers A and B separated by a space.

Output Format

Print the value of A^B .

Constraints

$1 \leq A \leq 100$ $0 \leq B \leq 20$

Logic

1. Start.
2. Read number n and power m.
3. Use a variable to store res.
4. Using a for loop iterate till m and multiply m with variable res.
5. Print the result.
6. Stop.

Source Code

```
import java.util.Scanner;

import static problemOne.ProblemOne.*;
import static problemTwo.ProblemTwo.*;

public class Main {
    public static void main(String[] args) {
        System.out.println("Enter n and m: ");
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int m = sc.nextInt();
        printNToThePowerM(n,m);
    }
}
```

```
package problemTwo;

public class ProblemTwo {
    public static void printNToThePowerM(int n,int m) {
        int res = 1;
        for (int i = 1; i <= m; i++) {
            res*=n;
        }
        System.out.println("Output: "+res);
    }
}
```

Output

```
Enter n and m:  
2 5  
Output: 32  
  
Process finished with exit code 0
```

Time Complexity

- $O(n)$
 - Using a for loop iterating till n and multiplying n, m times resulting in n time complexity

Space Complexity

- $O(1)$